

Verificação de uma classe de frações: geração aleatória guiada por feedback e teste baseado em propriedades

Eduardo Hoffmann, Lucas Mocelin e Fernando Kunst

Verificação e Validação – PUCRS, Escola Politécnica, 2026/I – Trabalho T1

Repositório: github.com/theeduardohoffmann/fraction-lab

1 Introdução

O kata escolhido é `Fraction`, um objeto de valor imutável que representa números racionais com numerador e denominador `long`, sempre normalizados: denominador positivo, fração irredutível e zero como `0/1`. A classe oferece criação (`of`, `parse`), aritmética (`plus`, `minus`, `times`, `dividedBy`), ordem e igualdade por valor, e sinaliza overflow com `ArithmeticException`. A implementação usa Java 17, Maven, JUnit 5, jqwik 1.9.3, Randoop 4.3.4 e JaCoCo, com integração contínua no GitHub Actions.

O objetivo é aplicar duas técnicas ao mesmo problema, compará-las criticamente e justificar a escolha: (1) geração aleatória guiada por feedback com contratos como oráculo e (2) teste baseado em propriedades. A classe é pequena, mas reúne invariantes fortes, uma álgebra bem conhecida (o corpo dos racionais) e um modo de falha numérico (overflow), o que expõe a diferença entre um oráculo genérico e propriedades do domínio.

2 Referencial

Pacheco et al. (2007) propõem a geração aleatória guiada por feedback, implementada no Randoop. Sequências de chamadas são montadas aos poucos e executadas, e o resultado de cada execução orienta a geração seguinte. O oráculo é um conjunto de contratos (a Figura 4 do artigo lista: `o.equals(o)` verdadeiro; `equals`, `hashCode` e `toString` sem exceção; nenhum `NullPointerException` quando nenhum argumento era nulo; nenhum `AssertionError`), de modo que ninguém precisa escrever o resultado esperado, e o usuário pode acrescentar contratos do domínio. Três filtros evitam entradas inúteis: o de igualdade descarta objetos iguais (por `equals`) a outros já criados, o de nulos impede reaproveitar `null` e o de exceções impede que sequências que lançam exceção sejam estendidas, pois a exceção costuma indicar violação de pré-condição. Para argumentos primitivos, o gerador sorteia valores de um conjunto fixo e pequeno (como `-1`, `0` e `1`), que o usuário pode ampliar. Pontos fortes: automação, baixo custo de especificação e boa detecção de falhas em bibliotecas reais (14 bibliotecas, 780 mil linhas). Limitações: contratos genéricos dizem pouco sobre a correção funcional; o filtro de exceções supõe que exceção significa entrada inválida; e o próprio artigo admite que o filtro de igualdade pode fazer a ferramenta perder erros, se objetos iguais se comportarem de modo diferente.

Fink e Bishop (1997) apresentam o teste baseado em propriedades como forma de assegurar que um programa está livre de falhas genéricas, especificadas formalmente. O analista escreve a propriedade em TASPEC; o programa é fatiado para isolar o código relevante; um monitor de execução, gerado da especificação, faz o papel de oráculo; e a métrica de cobertura de fluxo de dados *iterative contexts* indica quando o teste está completo. O estudo de caso é um defeito de autenticação no `ftpd`. Pontos fortes: critério explícito de completude e oráculo derivado da especificação. Limitações declaradas pelos autores: a geração de dados para as lacunas de cobertura não é automática, não há método para escolher as propriedades, e a validação vale apenas para as propriedades especificadas.

Comparação e evolução. Entre 1997 e 2007 o oráculo deixou de ser uma propriedade formal escrita para cada caso e passou a ser um conjunto de contratos genéricos aplicáveis a qualquer classe, e a geração de dados, que Fink e Bishop deixam ao testador, tornou-se automática. Ganha-se custo baixo; perde-se expressividade. Uma ressalva: o jqwik não implementa a técnica de Fink e Bishop (não há fatiamento nem *iterative contexts*). Ele gera entradas aleatórias a partir de geradores declarados e verifica propriedades algébricas. Do artigo aproveitamos a ideia central, a propriedade como especificação e oráculo, e a ligação com o jqwik é uma interpretação da equipe, não uma afirmação dos autores. Cabe notar que o próprio artigo de Pacheco et al. cita, entre os usos de teste aleatório, o QuickCheck para programas Haskell, ferramenta a que o jqwik se assemelha.

3 Aplicação

Randoop. Os contratos padrão valem sem configuração: C1, C5 e C6 vêm da Figura 4 do artigo, e a versão 4.3.4 verifica ainda simetria e transitividade de `equals`, `hashCode` consistente (C2–C4) e contratos de `compareTo`, que o artigo não lista. Registramos quatro contratos do domínio como pós-condições em JSON: denominador positivo (C7), irredutibilidade (C8), `compareTo` coerente com `equals` (C9) e ida e volta `parse(toString())` (C10). Como o conjunto padrão de primitivos é pequeno, fornecemos um arquivo de literais (± 1000 , ± 7 , `Long.MAX_VALUE` e `Long.MIN_VALUE`, entre outros). As execuções duram 60 s (o padrão descrito no artigo é de 2 minutos), com semente 42; os testes gerados (JUnit 4) rodam pelo motor *vintage*.

jqwik. Formulamos 14 propriedades (P1–P14): normalização, igualdade por valor, ida e volta, comutatividade, associatividade, elementos neutros, inversos, distributividade, ordem total, divisão e exceções. As propriedades algébricas usam geradores pequenos ($|n| \leq 1000$, $1 \leq d \leq 1000$) para isolar a álgebra do overflow. O overflow é tratado em propriedades separadas (P1, P10d, P13), com a faixa completa de long e uma referência em BigInteger: o resultado deve ser exato ou ArithmeticException, nunca um valor errado.

Baseline. 15 testes de exemplo por partição de equivalência e valor limite (EX-01...15), para contraste. O arquivo tests.md liga cada teste a técnica, contrato ou propriedade e requisito (R1–R15).

4 Discussão

Resultados. No código correto, os 15 exemplos e as 18 execuções de propriedades (1000 tentativas cada) passaram. O Randoop gerou 1.674 sequências e 884 testes de regressão, sem nenhuma violação de contrato (917 testes no total, 0 falhas). Em P13, de 3.949 operações verificadas, 86% não estouraram e 14% lançaram ArithmeticException; nenhuma devolveu valor errado.

Para medir o poder de detecção, injetamos cinco defeitos à mão. Consideramos "detectado" quando ao menos um teste da técnica falha (Randoop: semente 42, 60 s).

Defeito injetado	Exemplos	jqwik	Randoop
M1: sinal do denominador não normalizado	sim (EX-01)	sim (P1, P2, P3, P11)	sim (1 teste)
M2: sem redução por mdc	sim (4 testes)	sim (P1, P2, P5, P9, P11)	sim (6 testes; inclui C8)
M3: compareTo por multiplicação em long	não	só P10d (faixa larga)	sim (1 teste)
M4: overflow ignorado	sim (EX-12)	sim (P13)	não
M5: hashCode só do numerador	não	não	não

Tabela 1 – Detecção de defeitos injetados. Fonte: execuções da equipe (tests.md, seção 5).

Contratos genéricos × propriedades. M4 mostra o limite do oráculo genérico: nenhum contrato fala de aritmética, então o Randoop não a enxerga (0 testes de erro nas sementes 1, 2, 3 e 42), enquanto P13 a detecta porque compara com uma referência exata. A vantagem, porém, não é de um lado só. Em M3, as propriedades de faixa pequena (P10a–c) não detectam o defeito, e só P10d, de faixa larga, o faz. O Randoop o detectou por um contrato próprio de transitividade de compareTo, que o próprio Randoop aplica (as mensagens de falha mostram compareTo-transitive), provavelmente porque os literais extremos provocaram o estouro. A qualidade das entradas pesa tanto quanto o oráculo. M5 é um mutante equivalente em relação aos contratos: como a forma é normalizada, frações iguais mantêm o mesmo hash, e nenhum oráculo baseado em equals e hashCode o vê.

Custo de especificação. No Randoop bastaram quatro entradas JSON e um arquivo de literais; nas propriedades, 18 testes, geradores com faixas escolhidas e um oráculo em BigInteger, o que exige entender o domínio e o risco de overflow. O primeiro é barato e cego ao domínio; o segundo é caro e preciso.

Filtro de exceções. Divisão por zero, recíproco de zero e overflow lançam exceção por especificação, não por entrada inválida. Nas execuções do Randoop houve 61 execuções excepcionais contra 20,9 milhões normais, e as sequências excepcionais aparecem nos testes de regressão como "exceção esperada" (comportamento da versão 4.3.4). Conforme o artigo, o filtro impede que tais sequências sejam estendidas, então a região do overflow é pouco explorada. Além disso, o teste de regressão apenas congela o comportamento atual; quem confere que a exceção é a exigida pela especificação são P12, P13 e EX-04, EX-10 e EX-12.

Dificuldades. (i) A primeira versão da pós-condição C10 lançava exceção ao avaliar parse sobre texto malformatado, e o Randoop abortou; foi preciso uma guarda de formato. (ii) Com o conjunto padrão de primitivos o gerador usou quase só of(1, 1) e não detectou M1 numa execução de 20 s, o que nos levou a fornecer literais. (iii) Foi necessário decidir o tratamento de Long.MIN_VALUE (o numerador nunca pode assumi-lo) para que negate e abs sejam seguros. (iv) Mesmo com a mesma semente, o Randoop produz quantidades diferentes de sequências, porque limita a geração por tempo.

Ameaças à validade. *Interna:* os defeitos foram injetados à mão, são só cinco, e "detectado" significa apenas "algum teste falhou". *Externa:* uma única classe pequena e numérica. *De construção:* as mutações representam alguns tipos de defeito; as propriedades foram escritas pela mesma equipe que escreveu a classe, o que pode enviesar o resultado. *De conclusão:* poucas repetições (quatro sementes para M1 e M4, uma para os demais) e nenhum tratamento estatístico.

5 Conclusão

As técnicas se complementam. O Randoop custa pouco e verifica contratos de objeto e sequências de chamadas, mas não conhece a aritmética do domínio. As propriedades capturam a correção algébrica e o overflow, ao custo de

modelar o domínio e escolher bem os geradores. Aprendemos que (a) as entradas importam tanto quanto o oráculo (literais no Randoop, faixa nos geradores do jqwik); (b) um oráculo só é confiável depois de validado com defeitos injetados, pois um contrato pode não disparar e nunca saberíamos; e (c) nenhuma técnica detecta defeitos equivalentes aos contratos, como M5. A comparação histórica indica o mesmo compromisso: da propriedade formal de Fink e Bishop ao contrato genérico de Pacheco et al., ganha-se automação e perde-se precisão.

Referências

FINK, G.; BISHOP, M. Property-based testing: a new approach to testing for assurance. **ACM SIGSOFT Software Engineering Notes**, New York, v. 22, n. 4, p. 74-80, jul. 1997. DOI: 10.1145/263244.263267. Disponível em: <https://nob.cs.ucdavis.edu/bishop/papers/1997-sen>. Acesso em: 2 out. 2026.

PACHECO, C.; LAHIRI, S. K.; ERNST, M. D.; BALL, T. Feedback-directed random test generation. *In*: INTERNATIONAL CONFERENCE ON SOFTWARE ENGINEERING, 29., 2007, Minneapolis. **Proceedings** [...]. 2007. p. 75-84. Disponível em: <https://homes.cs.washington.edu/~mernst/pubs/feedback-testgen-icse2007.pdf>. Acesso em: 2 out. 2026.